



Examen #2

Índices y Mejoras en el Desempeño de Consulta

Guía de abastecimiento – PostgreSQL Aurora

Camilo José Ardila Restrepo

Escuela de Ingenierías, Universidad Pontificia Bolivariana

Tópicos Avanzados en Bases de Datos

Docente: Juan Darío Rodas Marín

7 de octubre de 2026

Requisitos:

- Cuenta AWS con acceso al panel “Aurora and RDS”
- IDE enfocado en bases de datos
- Tiempo

Notas:

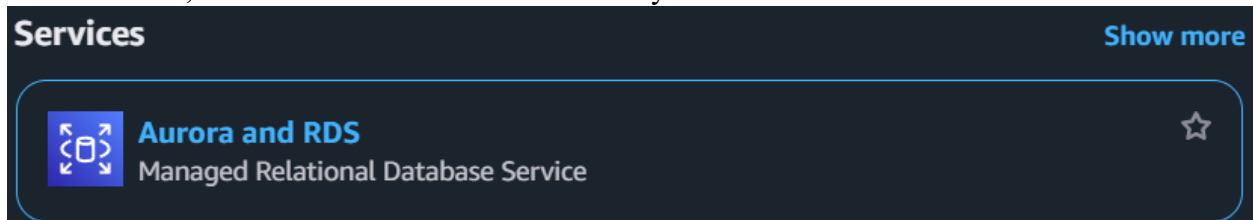
Para el propósito de esta práctica, se usó una cuenta free tier de AWS, dónde la realización de esta no implica ningún costo.

Usar un cluster en el servicio RDS puede ser overkill para una base de datos simple, sin embargo, es una herramienta relevante e importante de aprender.

El servicio de Aurora suele estar bloqueado en cuentas educativas, el free tier nos deja hacer uso de él, pero con configuraciones altamente limitadas.

1. Creación de la base de datos

Primeramente, accederemos a la consola de AWS y buscaremos el servicio “Aurora and RDS”:



Dentro de este servicio, seleccionaremos “Create” con configuración completa

A continuación, se muestran los parámetros usados para la instanciación de la base de datos, todos estos se llenarán dentro del panel de creación y nos permitirán configurar una infraestructura adecuada para el caso de uso que se le dará:

Engine options: PostgreSQL

Database creation method: Full Configuration

Templates: Free tier

Engine options

Engine type [Info](#)

☒ Aurora (MySQL Compatible)

☐ Aurora (PostgreSQL Compatible)

☐ MySQL

☒ PostgreSQL

☐ MariaDB

☐ Oracle

☐ Microsoft SQL Server

☐ IBM Db2

Choose a database creation method

☒ Full configuration
You set all of the configuration options, including ones for availability, security, backups, and maintenance.

☐ Easy create
Use recommended best-practice configurations. Some configuration options can be changed after the database is created.

Templates

Choose a sample template to meet your use case.

☐ Production
Use defaults for high availability and fast, consistent performance.

☐ Dev/Test
This instance is intended for development use outside of a production environment.

☒ Free tier
Use RDS Free Tier to develop new applications, test existing applications, or gain hands-on experience with Amazon RDS.

Availability and durability: Single-AZ ...

Availability and durability

Deployment options [Info](#)

Choose the deployment option that provides the availability and durability needed for your use case. AWS is committed to a certain level of uptime depending on the deployment option you choose. [Learn more in the Amazon RDS service level agreement \(SLA\)](#).

☐ Multi-AZ DB cluster deployment (5 instances)
Creates a primary DB instance with two readable standbys in separate Availability Zones. This setup provides:
• 99.99% uptime
• Redundancy across Availability Zones

☐ Multi-AZ DB instance deployment (2 instances)
Creates a primary DB instance with a non-readable standby instance in a separate Availability Zone. This setup provides:
• 99.99% uptime
• Redundancy across Availability Zones

☒ Single-AZ DB instance deployment (1 instance)
Creates a single DB instance without standby instances. This setup provides:
• 99.9% uptime
• No data redundancy

Settings

Engine version: PostgreSQL 18.3R2

RDS extended support: deshabilitado

Asimismo, le daremos un nombre a nuestra instancia y asignaremos un usuario y contraseña administrativos, estas serán las credenciales que usaremos para nuestra primera conexión.

Settings

Engine version [Info](#)

View the engine versions that support the following database features.

[Show filters](#)

Engine version

PostgreSQL 18.3-R2

☐ Enable RDS Extended Support [Info](#)
Amazon RDS Extended Support is a paid offering. By selecting this option, you consent to being charged for this offering if you are running your database major version past the RDS end of standard support date for that version. Check the end of standard support date for your major version in the [RDS for PostgreSQL documentation](#).

DB instance identifier [Info](#)

Type a name for your DB instance. The name must be unique across all DB instances owned by your AWS account in the current AWS Region.

database-tadb2

The DB instance identifier is case-insensitive, but is stored as all lowercase (as in "mydbinstance"). Constraints: 1 to 63 alphanumeric characters or hyphens. First character must be a letter. Can't contain two consecutive hyphens. Can't end with a hyphen.

Credentials Settings

Master username [Info](#)

Type a login ID for the master user of your DB instance.

postgres

1 to 16 alphanumeric characters. The first character must be a letter.

Credentials management

You can use AWS Secrets Manager or manage your master user credentials.

☐ Managed in AWS Secrets Manager - most secure
RDS generates a password for you and manages it throughout its lifecycle using AWS Secrets Manager.

☒ Self managed
Create your own password or have RDS create a password that you manage.

☐ Auto generate password
Amazon RDS can generate a password for you, or you can specify your own password.

Master password [Info](#)

Password strength: Very strong

(Lo más seguro, pero con implicaciones, es dejar que AWS maneje nuestras credenciales. Sin embargo, para entornos no productivos, se considera suficiente usar una contraseña fuerte)

Instance configuration

Instance type: db.t4g.micro

Storage type: General Purpose SSD (gp2) con 20GiB de espacio asignados

Instance configuration
The DB instance configuration options below are limited to those supported by the engine that you selected above.

DB instance class [Info](#)
▼ Hide filters

☐ Include previous generation classes

☐ Standard classes (includes m classes)

☐ Memory optimized classes (includes r and x classes)

☒ Burstable classes (includes t classes)

Instance type
db.t4g.micro
2 vCPUs 1 GiB RAM EBS Bandwidth: Up to 2085 Mbps Network: Up to 5 Gbps ▼

Storage

Storage type [Info](#)
Provisioned IOPS SSD (io2) storage volumes are now available.
General Purpose SSD (gp2)
Baseline performance determined by volume size ▼

Allocated storage [Info](#)
20 GiB
Allocated storage value must be 20 GiB to 6144 GiB

Connectivity

Compte resource: Don't connect to a ...

VPC: Create new VPC

DB Subnet group: Create new Subnet group

Security group: Create new (Recordar asignarle un nombre)

Connectivity [Info](#)

Compute resource
Choose whether to set up a connection to a compute resource for this database. Setting up a connection will automatically change connectivity settings so that the compute resource can connect to this database.

☒ **Don't connect to an EC2 compute resource**
Don't set up a connection to a compute resource for this database. You can manually set up a connection to a compute resource later.

☐ **Connect to an EC2 compute resource**
Set up a connection to an EC2 compute resource for this database.

Virtual private cloud (VPC) [Info](#)
Choose the VPC. The VPC defines the virtual networking environment for this DB instance.

Create new VPC
Generates a new VPC automatically when you submit the form.

After a database is created, you can't change its VPC.

DB subnet group [Info](#)
Choose the DB subnet group. The DB subnet group defines which subnets and IP ranges the DB instance can use in the VPC that you selected.

Create new DB Subnet Group

Public access [Info](#)

☒ **Yes**
RDS assigns a public IP address to the database. Amazon EC2 instances and other resources outside of the VPC can connect to your database. Resources inside the VPC can also connect to the database. Choose one or more VPC security groups that specify which resources can connect to the database.

☐ **No**
RDS doesn't assign a public IP address to the database. Only Amazon EC2 instances and other resources inside the VPC can connect to your database. Choose one or more VPC security groups that specify which resources can connect to the database.

VPC security group (firewall) [Info](#)
Choose one or more VPC security groups to allow access to your database. Make sure that the security group rules allow the appropriate incoming traffic.

☐ **Choose existing**
Choose existing VPC security groups

☒ **Create new**
Create new VPC security group

New VPC security group name
tadb2-sg

Monitoring: Database insights – Standard (Detailed database and per ...: Deshabilitado)

Monitoring [Info](#)
Choose the monitoring configuration for this database. The Standard mode of Database Insights offers seven days of retention for detailed database metrics at no additional charge. The Advanced mode offers fleet-level monitoring and premium features, including the long-term retention of database metrics. Database Insights pricing is separate from RDS monthly estimates. See [Amazon CloudWatch pricing](#).

☐ **Database Insights - Advanced**

- Fleet-level monitoring
- Integration with CloudWatch Application Signals
- Detailed database and per-query metrics (extended retention at no additional charge)

☒ **Database Insights - Standard**

- Detailed database and per-query metrics (free for 7 days of retention)
- Option to retain detailed metrics up to 24 months for an additional charge

☐ **Collect detailed database and per-query metrics**

Dejamos el resto de las configuraciones no especificadas por defecto y creamos el cluster

2. Garantizar una conexión cifrada

Comprobamos que el security group que hemos creado tenga el puerto 5432 abierto:

Edit inbound rules [Info](#)
Inbound rules control the incoming traffic that's allowed to reach the instance.

Inbound rules [Info](#)

Security group rule ID: sgr-0088fbbff6d613752

Type: PostgreSQL

Protocol: TCP

Port range: 5432

Source: Anywhere...

Description - optional:

Rules with source of 0.0.0.0/0 or ::/0 allow all IP addresses to access your instance. We recommend setting security group rules to allow access from known IP addresses only.

(Este puerto se abre debido a que, por defecto, solo se permite la ip pública actual, sin embargo, como tengo un dispositivo portátil, que cambia de ip pública activamente, es útil para evitar inconvenientes)

Asimismo, creamos un “Parameter group” dentro de Aurora, con el objetivo de forzar el uso de certificados SSL en la conexión:

Le ponemos un nombre y especificamos que es una PostgreSQL v18, de tipo “DB Parameter Group”:

Parameter group details

Parameter group name

pg-force-ssl

The name must have 1 to 255 characters and begin with a letter. The name can't end with a hyphen or contain two consecutive hyphens. Valid characters: A-Z, a-z, 0-9, and - (hyphen)

Description

Parameter group fara forzar conexiones cifradas

This description appears in the Parameter groups dashboard. You can use it to quickly identify the purpose of the parameter group.

Engine type

PostgreSQL

Parameter group family

postgres18

You can associate a DB parameter group with only one DB parameter group family. You can apply the parameter group only to a DB instance whose DB engine is compatible with the parameter group family.

Type

DB Parameter Group

Type for the DB parameter group

Parameters (456) Info						
Q rds.force_ssl 1 match						
Name	Value	Apply type	Data type	Value type	Source	
rds.force_ssl	1	Dynamic	Boolean	Modifiable	System default	

Primary storage

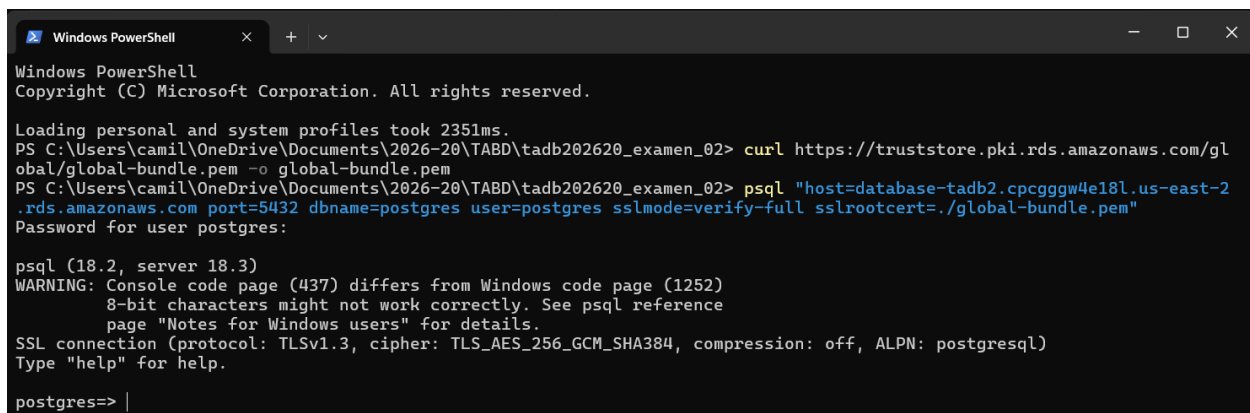
Encryption
Enabled

3. Conexión y carga de esquema y datos

Abrimos nuestra terminal en el repositorio “tadb202620_examen_02” (Clonar con “git clone https://github.com/Camilo-Ardila/tadb202620_examen_02” si no se tiene aún).

Estando dentro del repositorio, descargamos el bundle global de certificados de AWS usando “curl https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem -o global-bundle.pem”. Luego, lo copiamos manualmente al directorio “PostgreSQL-AWS\Scripts”.

Teniendo el certificado, haremos una prueba de conexión usando el ConnectionString, para esto, ejecutamos “psql "host=database-tadb2.cpcgggw4e18l.us-east-2.rds.amazonaws.com port=5432 dbname=postgres user=postgres sslmode=verify-full sslrootcert=./global-bundle.pem"”. Con esto, se nos pedirá la contraseña del usuario administrativo y podremos ver si la conexión a la base de datos es exitosa.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Loading personal and system profiles took 2351ms.
PS C:\Users\camil\OneDrive\Documents\2026-20\TABD\tadb202620_examen_02> curl https://truststore.pki.rds.amazonaws.com/global/global-bundle.pem -o global-bundle.pem
PS C:\Users\camil\OneDrive\Documents\2026-20\TABD\tadb202620_examen_02> psql "host=database-tadb2.cpcgggw4e18l.us-east-2.rds.amazonaws.com port=5432 dbname=postgres user=postgres sslmode=verify-full sslrootcert=./global-bundle.pem"
Password for user postgres:

psql (18.2, server 18.3)
WARNING: Console code page (437) differs from Windows code page (1252)
        8-bit characters might not work correctly. See psql reference
        page "Notes for Windows users" for details.
SSL connection (protocol: TLSv1.3, cipher: TLS_AES_256_GCM_SHA384, compression: off, ALPN: postgresql)
Type "help" for help.

postgres=> |
```

(Aunque el certificado es público, es útil guardarlo en un lugar seguro. En mi caso, como usé el repositorio del examen, aunque no es la mejor solución, simplemente se agregó al .gitignore)

A partir de ahora, correremos todos los comandos desde “PostgreSQL-AWS\Scripts”, que contiene todos los scripts .sql necesarios para el abastecimiento

Para la creación de la base de datos, corremos el siguiente comando:

```
psql "host=database-tadb2.cpcgggw4e18l.us-east-2.rds.amazonaws.com port=5432
dbname=postgres user=postgres sslmode=verify-full sslrootcert=./global-bundle.pem" -f
00_setup_database_and_roles.sql
```

```
PS C:\Users\camil\OneDrive\Documents\2026-20\TABD\tadb202620_examem_02\PostgreSQL-AWS\Scripts> psql "host=database-tadb2
2 dbname=postgres user=postgres sslmode=verify-full sslrootcert=./global-bundle.pem" -f 00_setup_database_and_roles.sql
Password for user postgres:
```

```
psql "host=database-tadb2.cpcgggw4e18l.us-east-2.rds.amazonaws.com port=5432
dbname=brechas_seguridad user=postgres sslmode=verify-full sslrootcert=./global-
bundle.pem" -f 01 create schema postgresql.sql
```

```
PS C:\Users\camil\OneDrive\Documents\2026-20\TABO\tadb202620_examen_02\PostgreSQL-AWS\Scripts> psql "host=database-tadb2.cpcgggw4
2 dbname=brechas_seguridad user=postgres sslmode=verify-full sslrootcert=./global-bundle.pem" -f 01_create_schema_postgresql.sql
Password for user postgres:

GRANT
SET
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE TABLE
CREATE INDEX
CREATE TABLE
CREATE INDEX
CREATE INDEX
CREATE TABLE
CREATE INDEX
CREATE INDEX
CREATE INDEX
CREATE INDEX
CREATE TABLE
CREATE INDEX
CREATE INDEX
CREATE INDEX
CREATE TABLE
CREATE INDEX
CREATE INDEX
CREATE INDEX
RESET
```

```
psql "host=database-tadb2.cpcgggw4e18l.us-east-2.rds.amazonaws.com port=5432
dbname=brechas_seguridad user=brechas_camilo sslmode=verify-full sslrootcert=./global-
bundle.pem" -f 02_load_data_postgresql.sql
```

Finalmente, creamos las vistas, procedimientos y lógica almacenada de utilidad para la práctica haciendo uso de su script correspondiente:

```
psql "host=database-tadb2.cpcgggw4e18l.us-east-2.rds.amazonaws.com port=5432
dbname=brechas_seguridad user=brechas_camilo sslmode=verify-full sslrootcert=./global-
bundle.pem" -f 03_funciones_vistas_postgresql.sql
```

```
Problems Output Debug Console Terminal Ports
PS C:\Users\camil\OneDrive\Documents\2026-20\TABD\tadb202620_examen_02\PostgreSQL-AWS\Scripts> psql "host=database-tadb2.cpcggw4e181.us-east-1.amazonaws.com dbname=tadb2 user=brechas_camilo sslmode=verify-full sslrootcert=../global-bundle.pem" -f 03_funciones_vistas_postgresql.sql
Password for user brechas_camilo:

psql:03_funciones_vistas_postgresql.sql:44: NOTICE:  function fn_dias_hasta_deteccion(pg_catalog.int4) does not exist, skipping
DROP FUNCTION
psql:03_funciones_vistas_postgresql.sql:45: NOTICE:  function fn_usuarios_afectados_brecha(pg_catalog.int4) does not exist, skipping
DROP FUNCTION
psql:03_funciones_vistas_postgresql.sql:46: NOTICE:  function fn_brecha_expuso_dato_critico(pg_catalog.int4) does not exist, skipping
DROP FUNCTION
psql:03_funciones_vistas_postgresql.sql:47: NOTICE:  function sp_brechas_criticas_ultimo_anio(date) does not exist, skipping
DROP FUNCTION
psql:03_funciones_vistas_postgresql.sql:48: NOTICE:  function sp_historial_exposicion_usuario(pg_catalog.varchar) does not exist, skipping
DROP FUNCTION
psql:03_funciones_vistas_postgresql.sql:49: NOTICE:  view "v_exposicion_detalle" does not exist, skipping
DROP VIEW
psql:03_funciones_vistas_postgresql.sql:50: NOTICE:  view "v_brecha_detalle" does not exist, skipping
DROP VIEW
SET
CREATE FUNCTION
CREATE FUNCTION
CREATE FUNCTION
CREATE FUNCTION
CREATE FUNCTION
CREATE VIEW
CREATE VIEW
RESET
      codigo_brecha | dias_hasta_deteccion | usuarios_afectados | expuso_dato_critico
-----
BR-00001           |          193         |          47         | t
(1 row)
```

(Si vemos, los últimos dos comandos se corrieron desde el usuario con privilegios mínimos “brechas_camilo”, quién solo posee los permisos para realizar las operaciones relevantes para esta práctica)

Con esto, nuestra base de datos queda lista para conectarnos desde un IDE y realizar las consultas y análisis relevantes.